EVALUATION & RESAMPLING

Model validation, cross-validation & the bootstrap

Elsa Ferreira Gomes

2026 / 2027

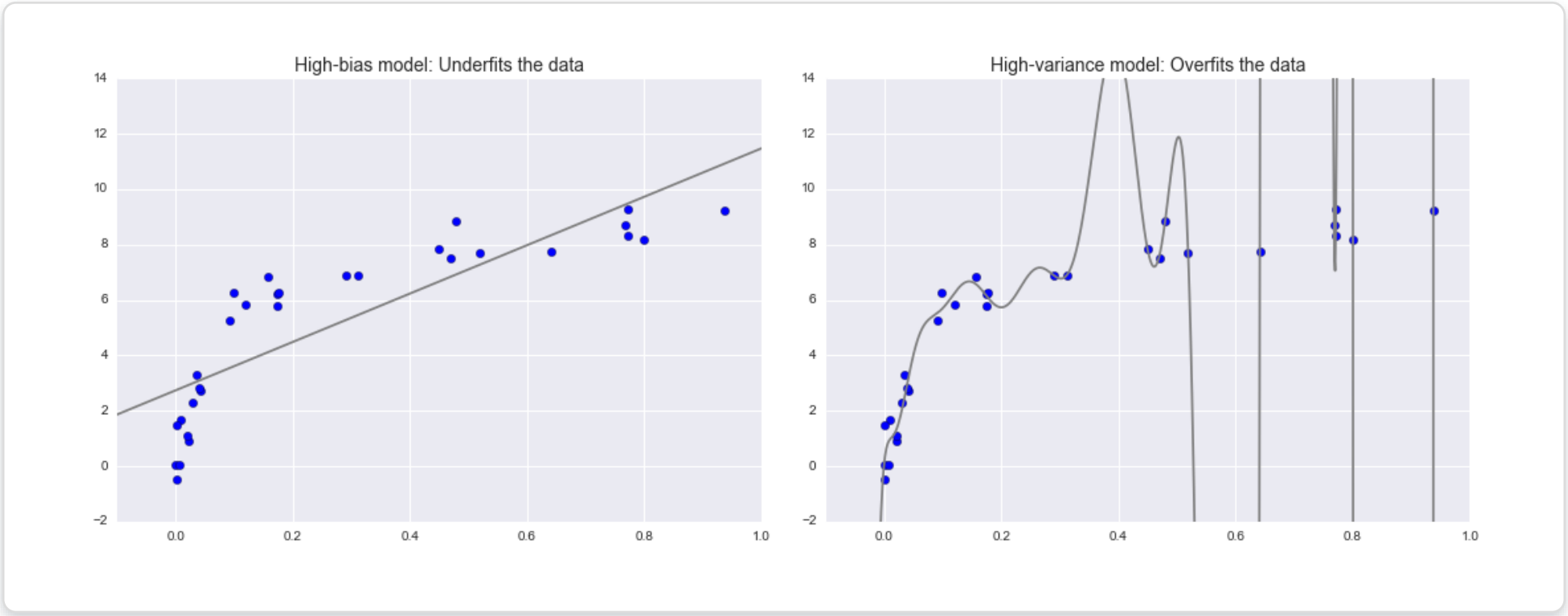
The basic supervised learning recipe

1. Choose the model
2. Choose model hyperparameters
3. Fit the model to the training data
4. Use the model to predict labels for new data

The missing piece: to make an informed choice, we need a way to validate that our model and hyperparameters are a good fit to the data. That's what this whole lecture is about.

The Bias-variance trade-off: finding the sweet spot

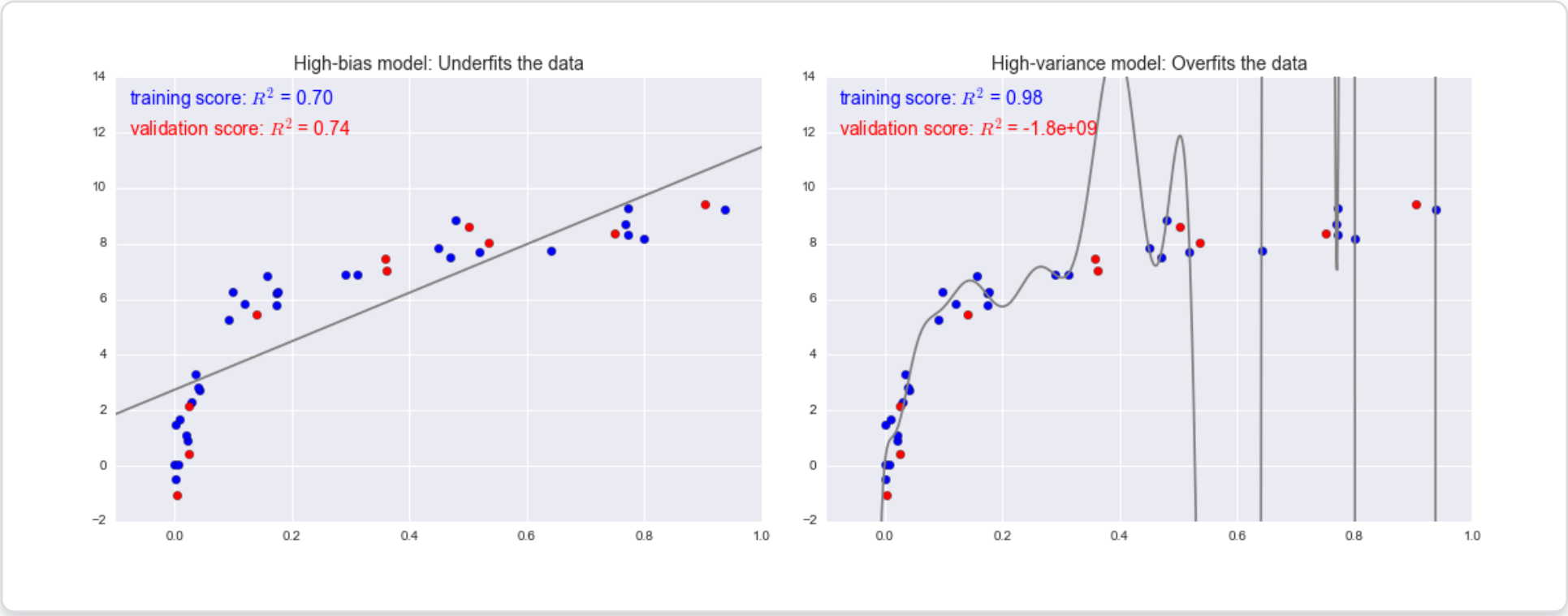
Two regression fits to the same dataset, ***both fail, but in different ways****:*



Left (underfit): a straight line can't capture the data's real complexity -> **high bias**. **Right (overfit):** a high-order polynomial fits the noise, not just the signal-> **high variance**

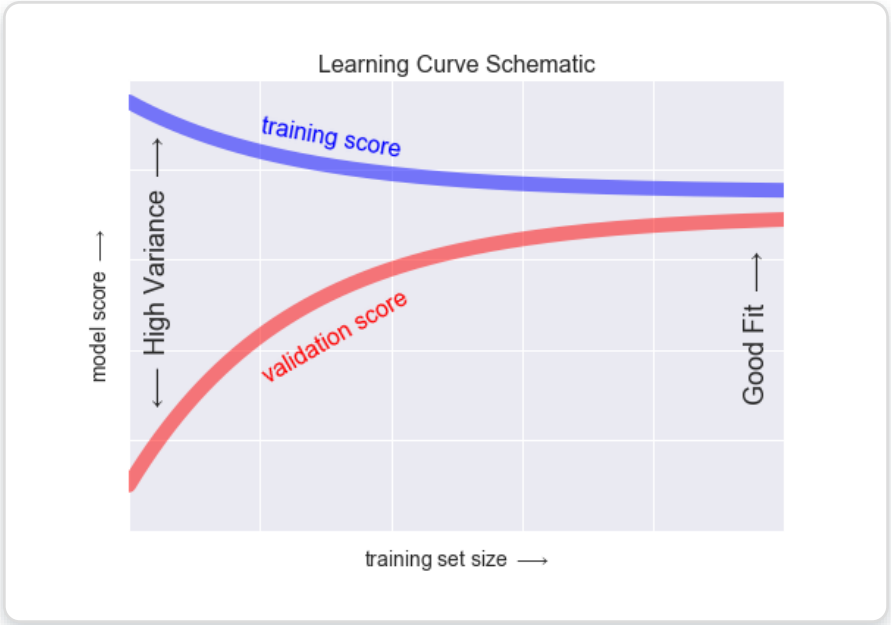
The Bias-variance trade-off: what happens on new data?

Red/lighter points were held out from training — the score shown is R^2 (coefficient of determination):



High-bias models: validation performance ≈ training performance. **High-variance models:** validation performance is far worse than training performance.

The validation curve



- Training score is always higher than validation score
- Low complexity (high bias): under-fit, poor on both training & new data
- High complexity (high variance): over-fits training, fails on new data
- The sweet spot is where the validation curve peaks

Resample Methods: why resample the data?

Resampling methods repeatedly draw samples from a training set and refit the model on each sample, to obtain additional information about the fitted model.

Example: estimating the variability of a linear regression fit

1. Repeatedly draw different samples from the training data
2. Fit a linear regression to each new sample
3. Examine the extent to which the resulting fits differ

This gives us information that fitting the model only once, on the original sample, simply cannot provide.

Resampling: two main approaches

Resampling can be computationally expensive (fitting the same method many times on different subsets) but modern computing power generally makes this feasible.

C

Cross-validation

Refits the model to samples formed from the training set, providing estimates of test-set prediction error.

B

Bootstrap

Resamples with replacement to quantify the uncertainty (standard deviation, bias) of a parameter estimate.

What are we actually trying to do?

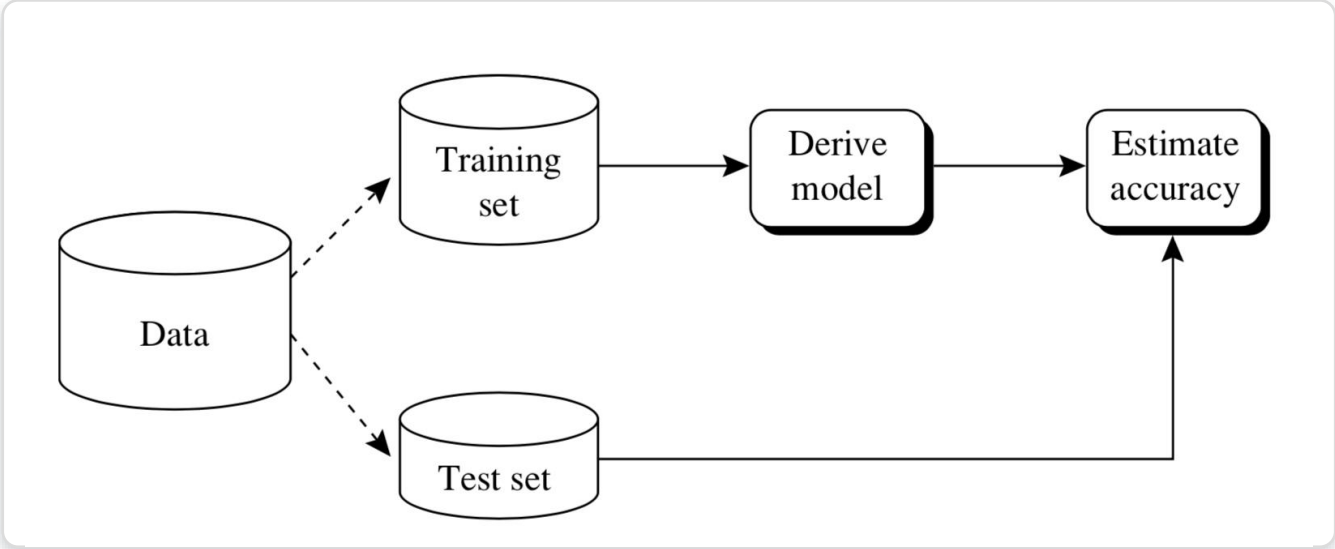
Goal: estimate how well the model performs with unknown cases: makes good predictions, good diagnoses, correctly assigns an email to a folder...

Model Selection

Example: is Logistic Regression better than Linear Discriminant Analysis?

1. Obtain a model with each approach (on the same data)
2. Identify the model that performs better (on the same data)

Two ways to estimate accuracy



Option 1 - training examples: optimistic, a **very bad choice**.

Option 2 - holdout: more reliable, but pessimistic and depends on sampling and set sizes.

Training error vs. generalization error

Training error: calculated on the observations used to train the model.
Test (generalization) error: the average error on a new, unseen observation.

Quantity	Formula	Meaning
Training Error	$\mathrm{err}_{training} = \frac{1}{N}\sum_{i=1}^{N} L(y_i, \hat{f}(x_i))$	average loss over the training sample
Test Error (given T)	$Err_T = \mathrm{E}[L(y_i, \hat{f}(x_i)) \mid T]$	expected prediction error over an independent sample given a training set T
Expected Test Error	$Err = \mathrm{E}[L(y_i, \hat{f}(x_i))]$	training set itself is not fixed

The aim of evaluation is to estimate Generalization Error as reliably (robustly) as possible from observed data.

CLASSIFIER METRICS

Metrics for evaluating classifier performance

How to assess a classifier? There are many metrics (these are the most popular):

Accuracy	**Recall**	**Precision**
F1	**Sensitivity**	**Specificity**

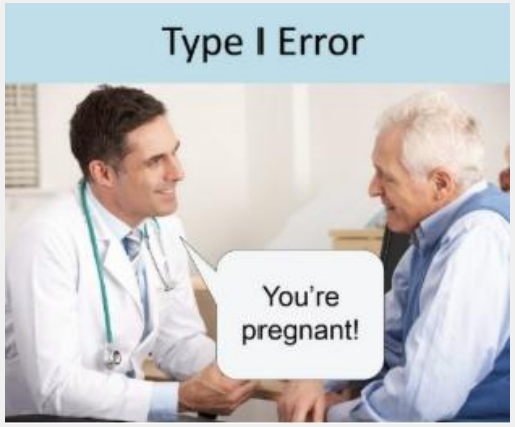
We'll define each of these precisely over the next few slides, with a worked example.

Two different kinds of error

Errors are not always equally important
- which side do you want to err on for a medical test?

False Positive

The model predicts positive,
while the true label is negative.



False Negative

The model predicts negative,
while the true label is positive.



The confusion matrix

Depending on how a model classifies a test case, four outcomes are possible:

	Predicted: Positive	**Predicted: Negative**
Actual: Positive	**TP** *True Positives*	**FN** *False Negatives*
Actual: Negative	**FP** *False Positives*	**TN** *True Negatives*

WORKED EXAMPLE

An example — loan approval

Loan companies profit from interest, but risk approving loans that won't be repaid:

classified as	Loan	no loan
loan	43	12
no loan	32	21

Accuracy: (43+21)/108 = 0.59

Recall (loan = positive): 43/(43+12) = 0.78 (catches 78% of good clients)

Precision: 43/(43+32) = 0.57 (57% of approved loans would fail)

Specificity: 21/(21+32) = 0.40 (only 40% of bad clients detected)

F1: 2×0.78×0.57/(0.78+0.57) = 0.66 (a decent balance)

METRICS · FORMULAS

Accuracy & Recall

```
Accuracy = (TP + TN) / (P + N)
```

The main diagonal divided by the sum of the whole matrix.

```
Recall = TP / P
```

a.k.a. True Positive Rate or Sensitivity: *how sensitive is the model to the positive class?*

METRICS · FORMULAS

Precision

Are all our “Yes” decisions good? If Precision = 1, we only say right things.

```
Precision = TP / (TP + FP)
```

- **The classic tension:** management wants higher **Recall** (more clients).
- Data scientists say: “we risk approving more bad clients too”.
- When **Recall increases**, **Precision tends to go down**, and vice-versa.

METRICS · FORMULAS

F1-score

How to combine recall and precision?
Calculate the harmonic mean:

$$F_1 = (2 \times \text{Recall} \times \text{Precision}) / (\text{Recall} + \text{Precision})$$

- Also known as F-score
- Low if either recall or precision is low
- Equal to Recall and Precision when Recall = Precision
- Generalised by Fβ (weights recall vs. precision differently)

Specificity

Are we excluding all the bad clients?
If Specificity = 1, we identify all of them (and hopefully some good ones too).

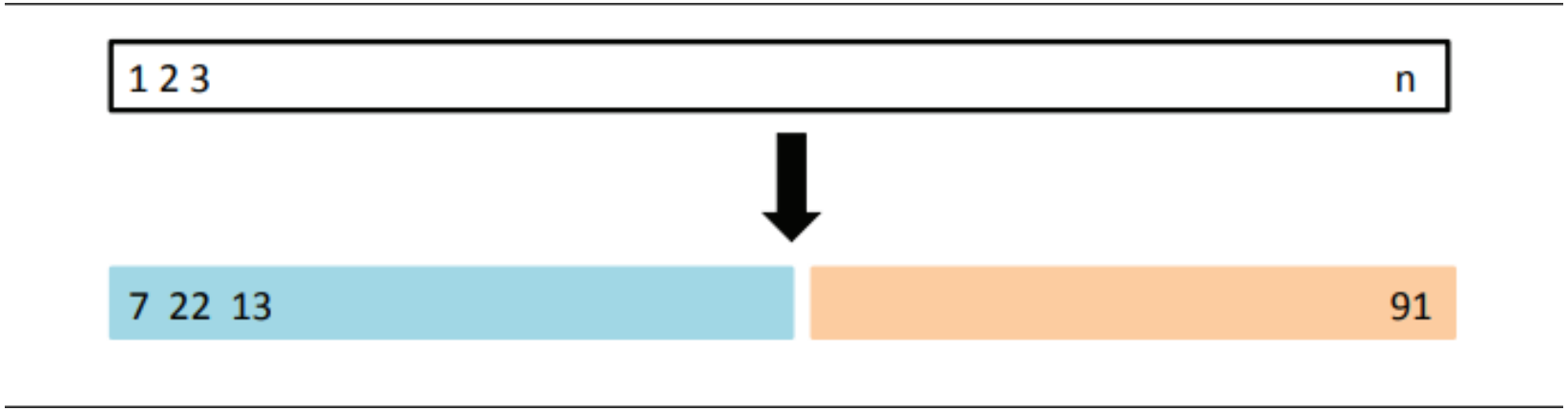
$$\text{Specificity} = TN / N$$

a.k.a. True Negative Rate *widely used in medical applications, where negatives are patients without the disease.*

The holdout evaluation method

Holdout: separate the data set into **train** and **test**.

Use **train to learn** the model and **test to assess** it.



A single split: one part of the data trains the model, the other part tests it — simple, but its result depends heavily on exactly how that one split happened to fall.

HOLDOUT

Dealing with assessment variance

A problem with holdout: the evaluation estimate itself has **variance**
- accuracy is a random variable,
- and we don't know its true value or distribution.

Test with more data

- Reduces variance
- but only works if there is more data available.

Repeat the train-test cycle

- Reduces variance
- Enables to study the distribution of the measure
- but needs a different sample each iteration.

K-fold cross-validation

- Widely used approach for estimating test error
- Used both to select the best model, and to estimate the test error of the final chosen model.

1. Randomly divide the data into K equal-sized parts
2. Leave out part k, fit the model to the other K−1 parts
3. Obtain predictions for the left-out k-th part
4. Repeat for each part k = 1, 2, …, K, then combine the results

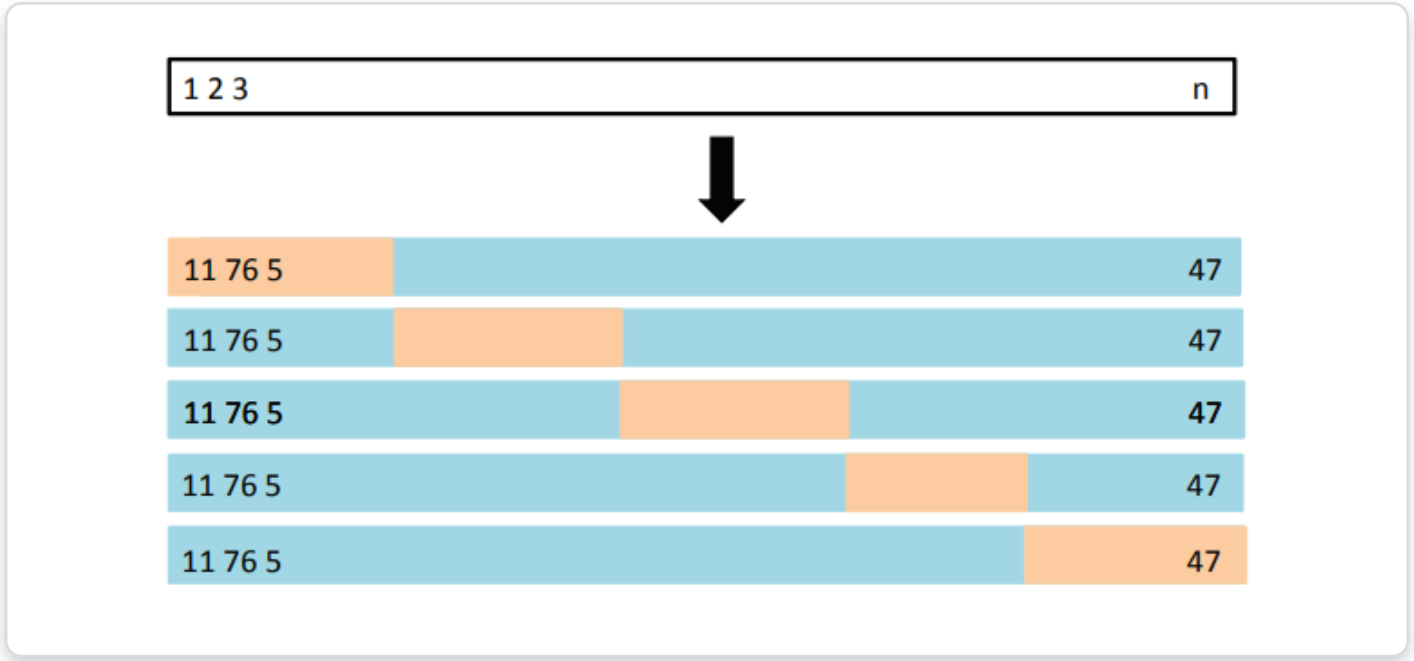
Repeating train-test, fold by fold

In each iteration $i \in \{1, \ldots, k\}$:

- Use fold i for testing
- Use the other k−1 folds for training
- Obtain Acc_i

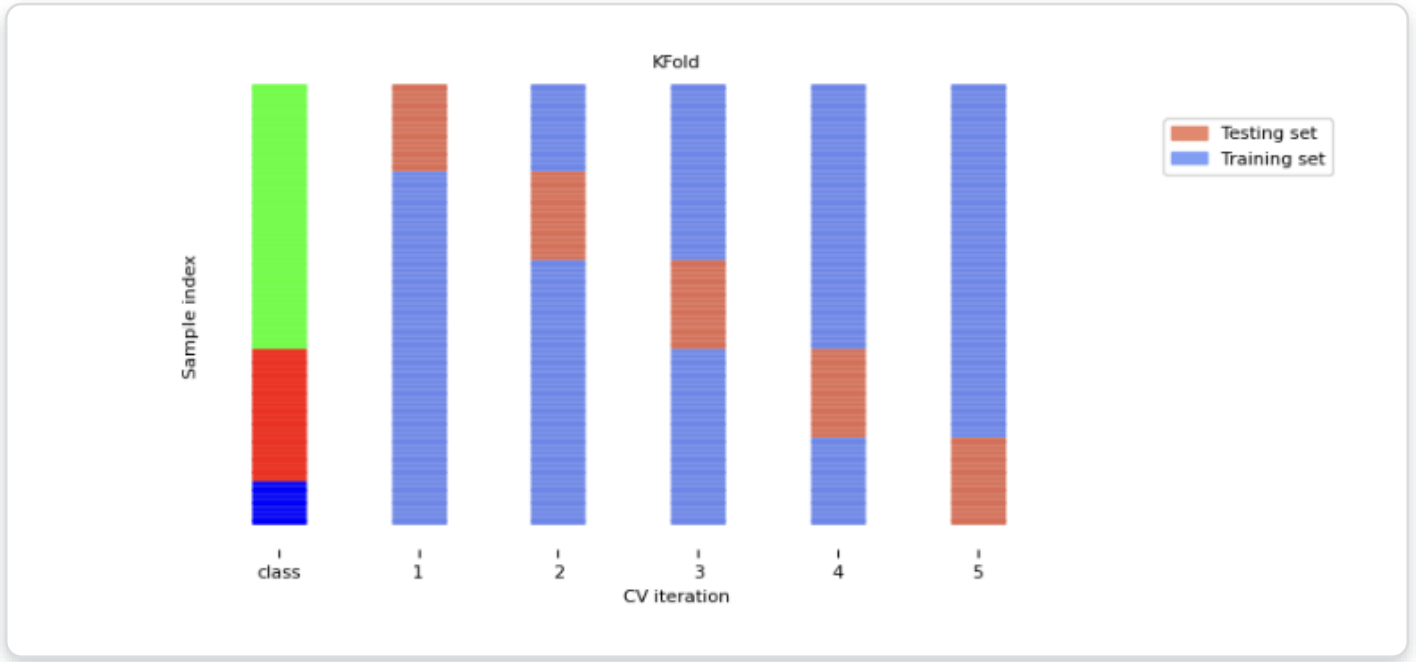
Then study the distribution of the k values Acc_i (mean, variance) to get a robust performance estimate.

5-fold CV, visualized



Each colored block is a fold. Across the 5 iterations, every fold takes a turn as the test set, so, every observation gets tested exactly once.

5-fold CV, visualized

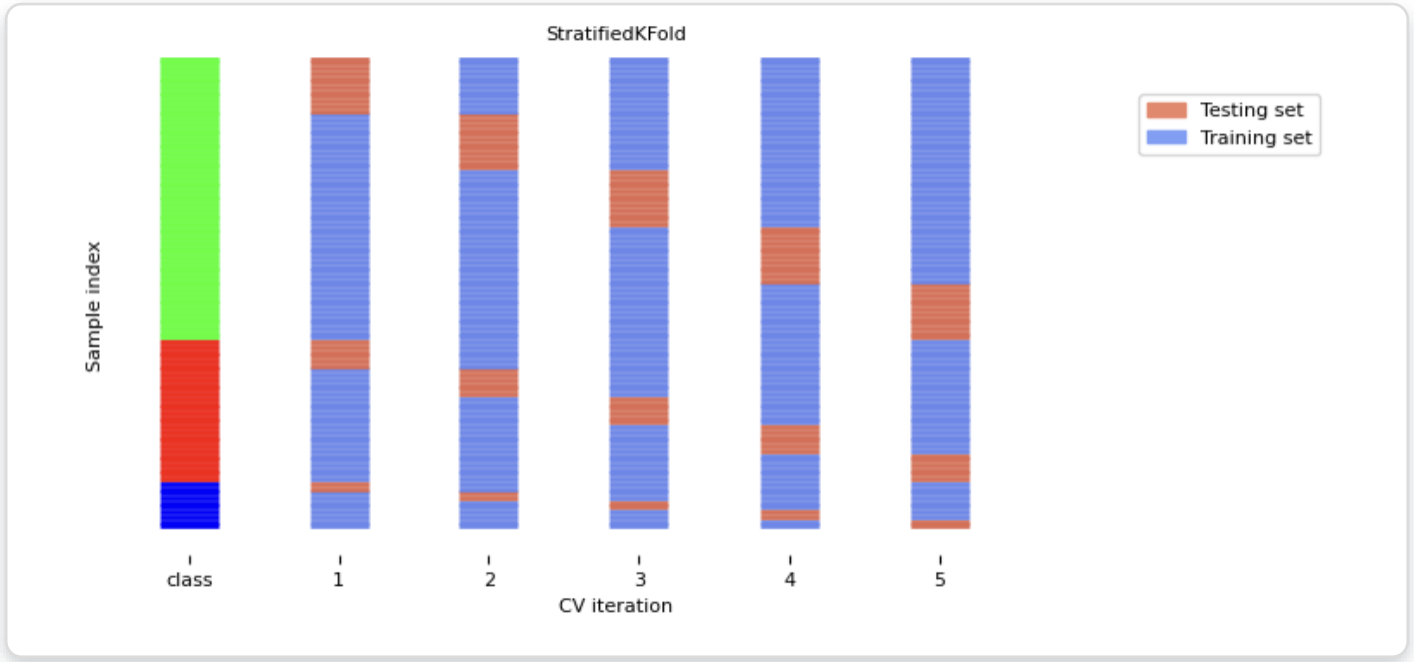


Each colored block is a fold. Across the 5 iterations, every fold takes a turn as the test set, so, every observation gets tested exactly once.

Practical properties of k-fold CV

Question	Guidance
Sample usage	Each sample used exactly once; test sets are independent (training sets are not)
Small class?	Use stratified cross-validation to avoid under-representation
Very few examples?	Use leave-one-out (or leave-p-out) cross-validation
How many folds?	Typical: 10-fold CV; Demsar (2006) suggests 2×5-fold CV
Before splitting	Shuffling the data first is usually a good idea

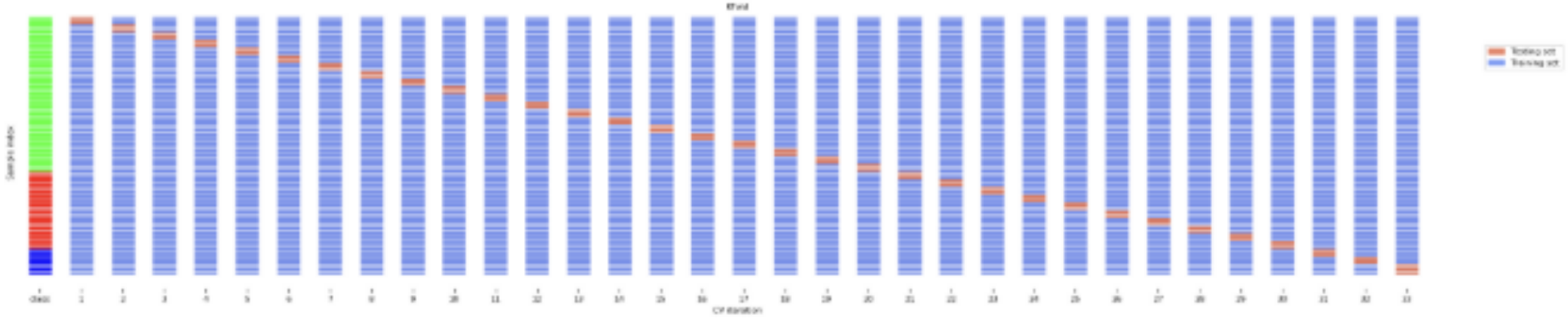
Stratified K-Fold cross-validation



*If the data is **unbalanced**, some classes could have only few samples.*
***Stratification**: proportions between classes are conserved in each fold*

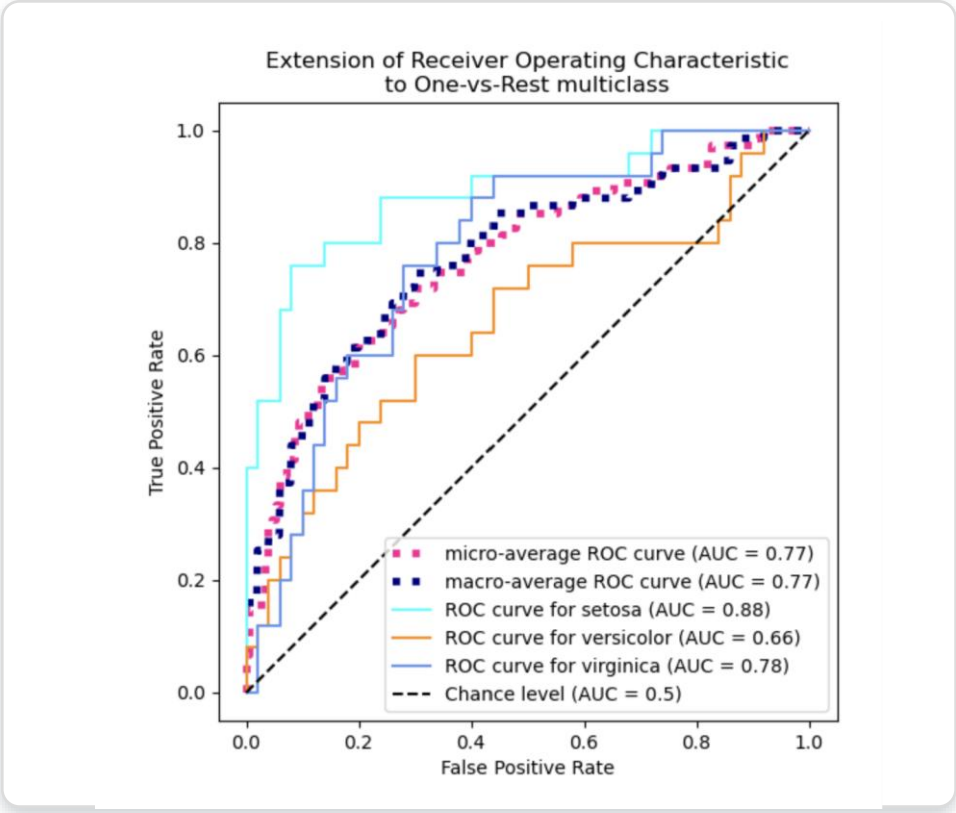
Leave-One-Out Cross-Validation (LOOC)

- If the sample is very small, we can **make K = N**
- We train with **all examples but one**, in each iteration
- Leave-one-out still estimates the Expected Test Error



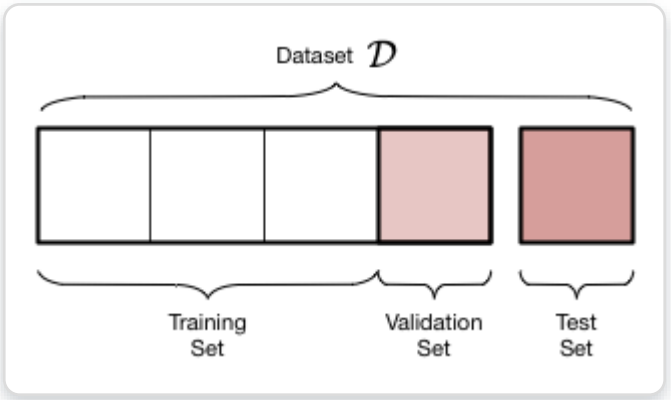
Comparing classifiers across thresholds

Receiver Operating Characteristic curves plot the relation between TPR and FPR across every possible misclassification cost:



AUC (Area Under the Curve) is the assessment measure — the closer to 1, the better the classifier across all thresholds.
(https://scikit-learn.org/stable/auto_examples/model_selection/plot_roc.html)

Why do we need both a validation and a test set?



- Cross-validation is used for **tuning**
  - hyperparameters, pre-processing decisions...
- After tuning (and other decisions), evaluate on a fresh, untouched **test set**
- The **internal test set** used during tuning is really the **validation set**

Using the test set correctly

Using the test set to improve results leads to overly optimistic results

- it's like using the future to make predictions,
- or knowing the exam questions when you study.

The test set should be for testing only.

Which model do we use in deployment?

- Use the approach and hyperparameters that had the best test results
- We can then use the whole data to train the final model (or less, if data is scarce)

Is the difference actually important?

Comparing algorithms A (0.8832 accuracy) and B (0.8845 accuracy)
- is this difference important?

Statistical significance

- Does the difference occur most of the time?
- How likely is it to observe this difference (or larger) by chance?

Usefulness

- Statistically significant does not imply useful.
- Does it save more lives, with fewer side effects?

A 10-fold CV example, with a t-test

1. Obtain two samples of accuracies: Acc_A and Acc_B (each of size 10)
2. Calculate the means, mean_A and mean_B
3. Assume accuracy values follow a t-distribution with k−1 degrees of freedom
4. Use a paired t-test
5. H_o (null hypothesis): the difference of means is zero
6. The paired t-test checks if we can reject H_o

Caveat: the test assumes independence of the samples (which, with cross-validation folds, isn't really true).

Choosing the right significance test

Situation	Recommended test
2 algorithms, 1 dataset (CV)	t-test — acceptable if data is large enough to promote independence
2 algorithms, many datasets (e.g. 30)	Parametric: t-test (if assumptions hold). Non-parametric: Wilcoxon signed-rank
Many algorithms, many datasets	Friedman test, then post-hoc Nemenyi with critical distances
Multiple comparisons	Be careful — sometimes p-values need adjusting

Rule of thumb: t-test with plain cross-validation should generally be avoided
- the folds' independence assumption doesn't truly hold, though it still gives useful signal.

CASE STUDY

Cross-validation in practice

Using the **Auto** dataset, we compare linear vs. higher-order polynomial terms in a linear regression - with the Validation Set Approach, Leave-One-Out CV, and k-fold CV.

Loading the Auto dataset

PYTHON

```
import sklearn.linear_model as skl_lm
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import (
    train_test_split, LeaveOneOut,
    KFold, cross_val_score)
from sklearn.preprocessing import (
    PolynomialFeatures)

df1 = pd.read_csv('Auto.csv',
    na_values='?').dropna()
df1.info()
```

OUTPUT

```
Index: 392 entries
Data columns (total 9):
 mpg           float64
 cylinders     int64
 displacement  float64
 horsepower    float64
 weight        int64
 acceleration  float64
 ...
```

392 observations (5 rows with missing horsepower dropped). We'll compare polynomial degrees 1 through 10 of horsepower, predicting mpg.

The Validation Set Approach

Randomly split the 392 observations into two sets: training (196) and validation (196):

PYTHON

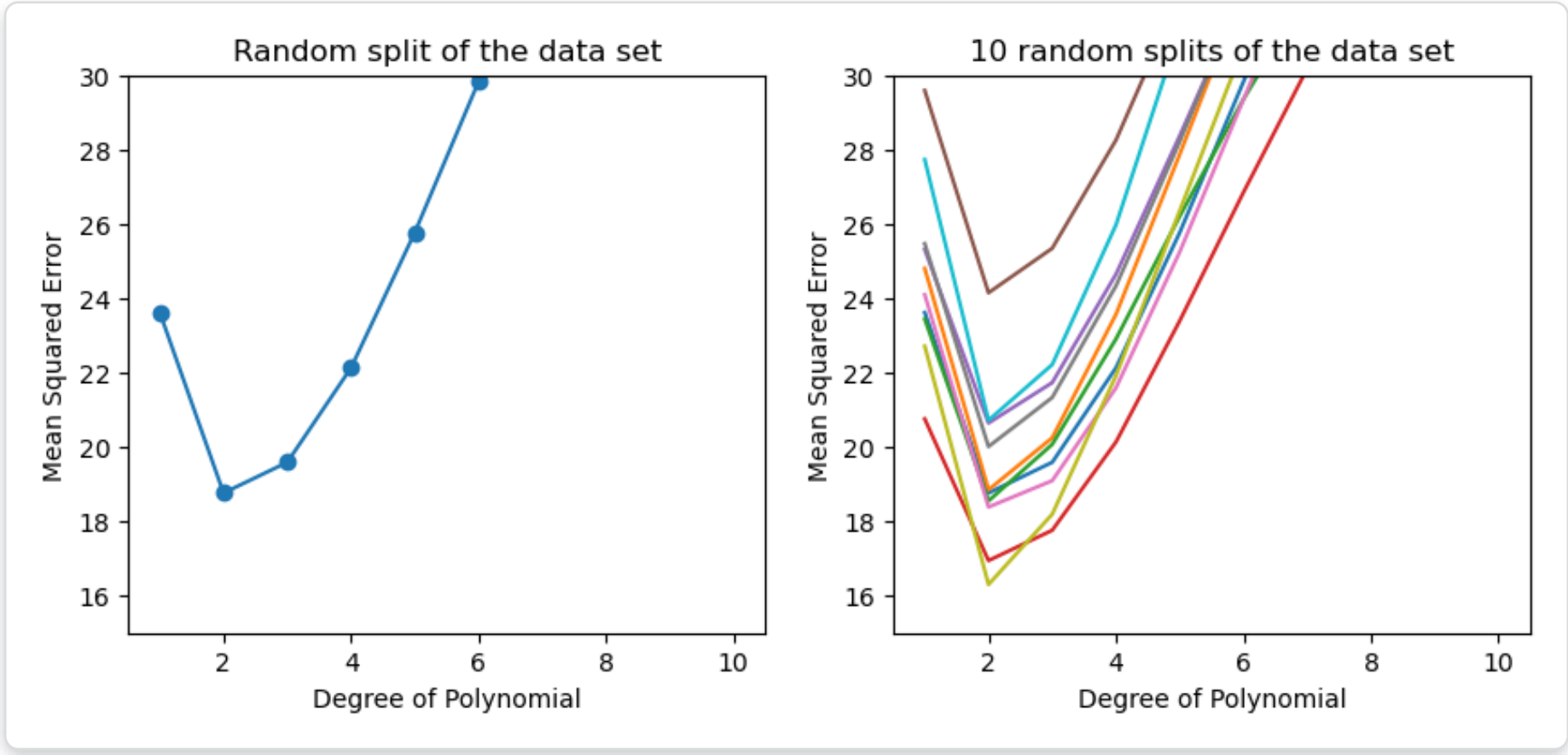
```
p_order = np.arange(1,11)
r_state = np.arange(0,10)

for (i,j),v in np.ndenumerate(Z):
    poly = PolynomialFeatures(X[i,j])
    X_poly = poly.fit_transform(df1.horsepower.values.reshape(-1,1))

    X_train,X_test,y_train,y_test=train_test_split(X_poly, df1.mpg,test_size=0.5, random_state=Y[i,j])

    regr.fit(X_train, y_train)
    pred = regr.predict(X_test)
    Z[i,j] = mean_squared_error(y_test, pred)
```

The validation set approach, visualized



Left: a single random split. **Right:** 10 different random splits — notice how much the "best" polynomial degree varies between splits. That variability is exactly the weakness of a single holdout split.

LOOCV and 10-fold CV in scikit-learn

PYTHON

```
# Leave-One-Out CV
regr = skl_lm.LinearRegression()
loo = LeaveOneOut()
scores = []
for i in p_order:
    poly = PolynomialFeatures(i)
    X_poly = poly.fit_transform(
        df1.horsepower.values.reshape(-1,1))
    score = cross_val_score(regr, X_poly,
        df1.mpg, cv=loo,

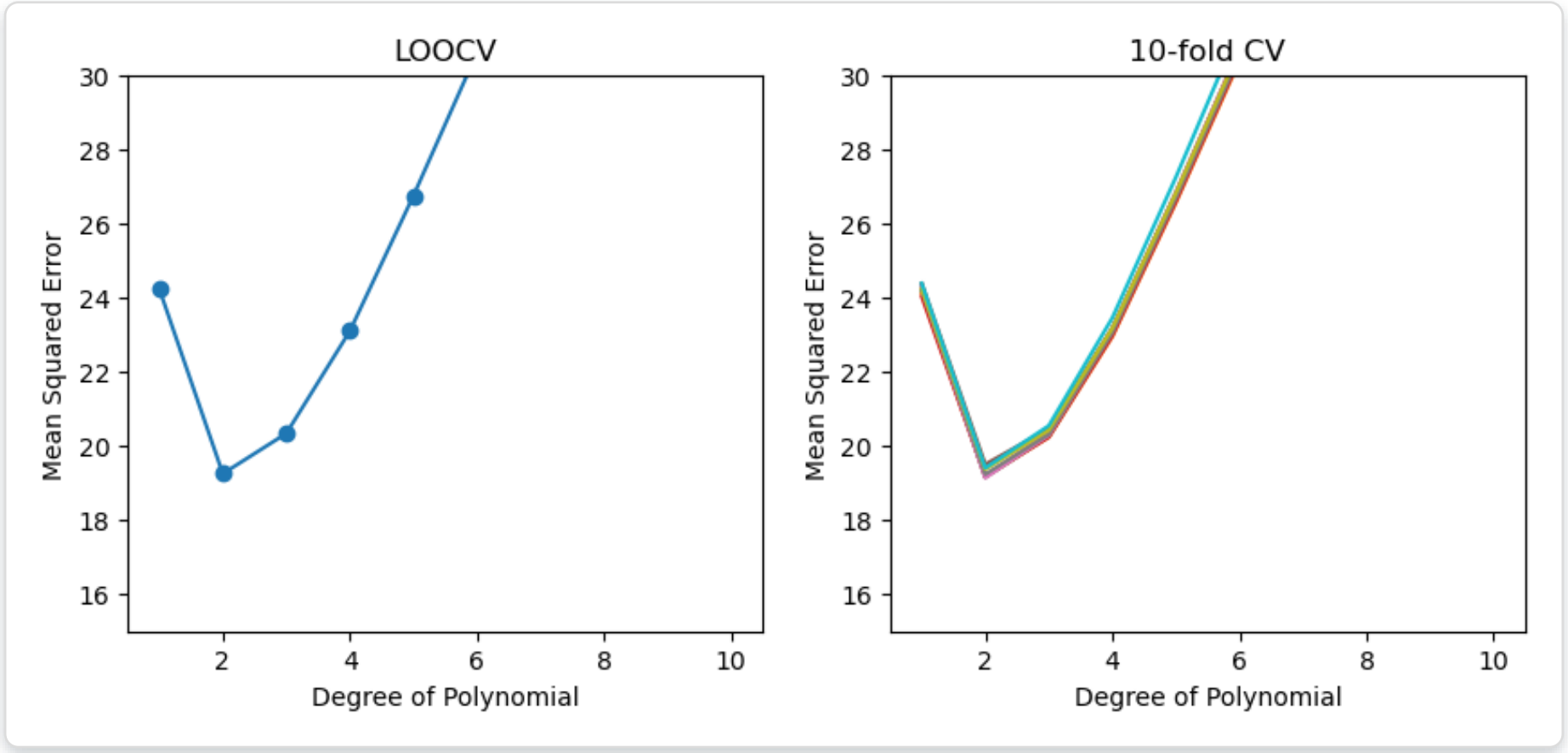
scoring='neg_mean_squared_error').mean()
    scores.append(score)
```

PYTHON

```
# 10-fold CV
folds = 10
for (i,j),v in np.ndenumerate(Z):
    poly = PolynomialFeatures(X[i,j])
    X_poly = poly.fit_transform(
        df1.horsepower.values.reshape(-
1,1))
    kf_10 = KFold(n_splits=folds,
        random_state=Y[i,j], shuffle=True)
    Z[i,j] = cross_val_score(regr, X_poly,
        df1.mpg, cv=kf_10,

scoring='neg_mean_squared_error').mean()
```

LOOCV vs. 10-fold CV, compared



Both methods agree: a quadratic term (degree 2) captures almost all the improvement — beyond that, MSE barely changes. But 10-fold CV (right) is far cheaper to compute than LOOCV (left, N=392 fits).

Choosing K in k-fold CV

K = N (LOOCV)

- Approximately unbiased estimator of Err.
- High variance.
- Computationally expensive.

K = 5

- Lower variance.
- Bias can be a problem:
  - smaller training sets,
  - easier to overfit,
  - error can be overestimated
  - less of an issue with enough data.

5 or 10-fold CV are recommended in general.

Quantifying uncertainty with the Bootstrap

- A powerful tool used to quantify the uncertainty associated with a given model or estimate
- It should not be confused with “booting” a computer, although the origins of the two terms are similar.

Example: estimating the average age of 1000 employees

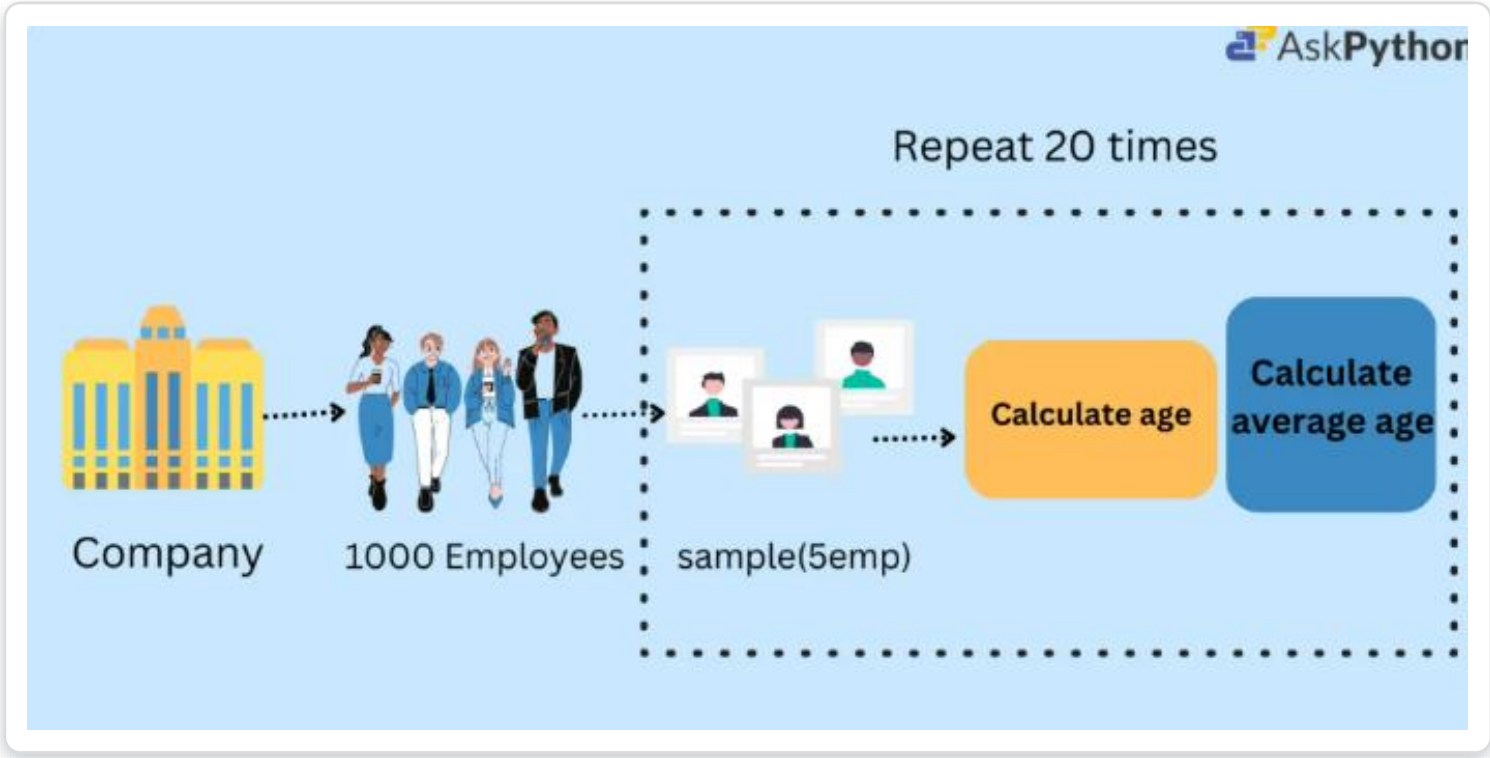
Ask everyone

- Survey all 1000 employees and calculate the mean age
- accurate, but time-consuming.

Bootstrap sampling

- Sample 5 employees, and collect their ages
- repeat 20 times, average the 100 collected ages
- a fast estimate of the population mean.

Bootstrap sampling, visualized



(https://www.askpython.com/python/examples/bootstrap-sampling-introduction)

Each bootstrap sample is drawn with replacement from the original data (the same size as the original, but some points repeat and others are left out).

Bootstrapping the mean age, in code

PYTHON

```
ages = [25, 30, 35, 40, 45, 50, 55, 60, 65, 70]

num_samples = 1000
bootstrap_means = np.zeros(num_samples)

for i in range(num_samples):
    sample = np.random.choice(ages, size=len(ages),replace=True)
    bootstrap_means[i] = np.mean(sample)

estimated_mean = np.mean(bootstrap_means)
estimated_std = np.std(bootstrap_means, ddof=1)

print("Estimated population mean age:", estimated_mean)
print("Standard error of the estimate:", estimated_std)
```

RESULT

```
Estimated population
mean age: 47.512

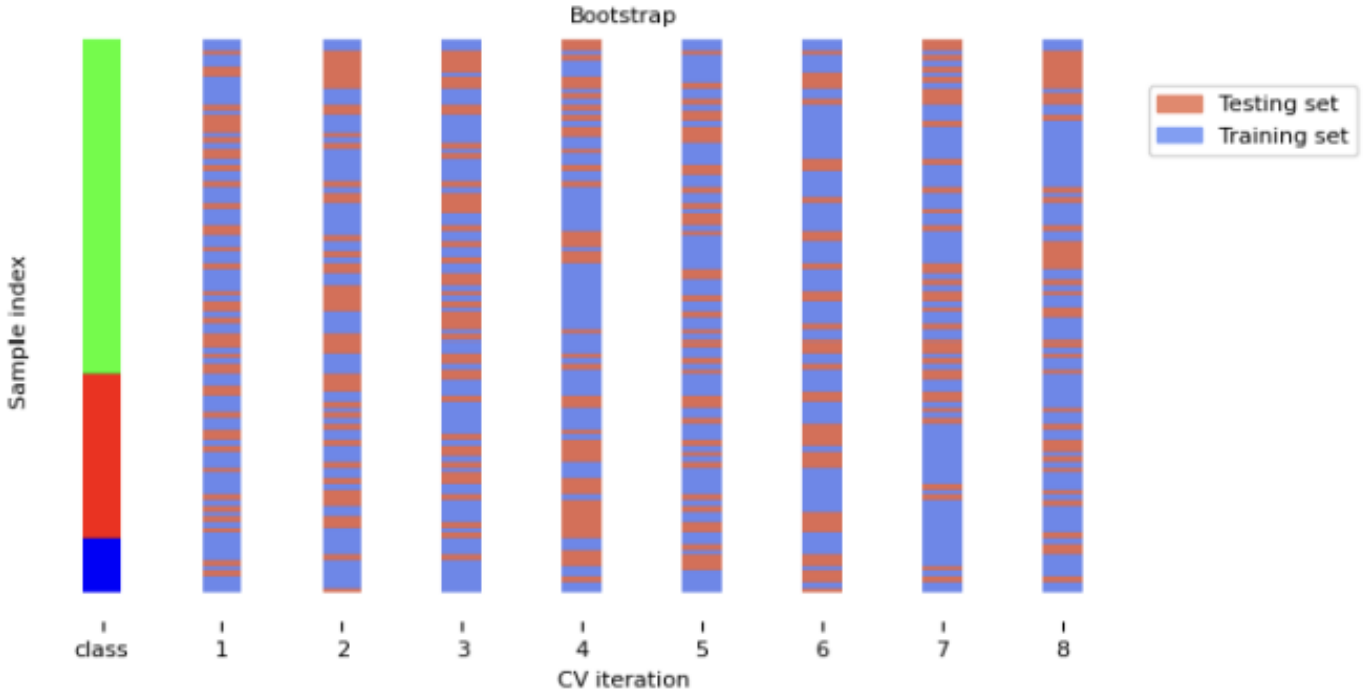
Standard error of
the estimate: 4.580
```

1000 resamples of these 10 ages (with replacement) give us not just a mean estimate, but also its standard error, a sense of how much that estimate could vary.

Bootstrapping as a resampling method

Bootstrapping samples the data set with replacement:

- ***k*** bootstrap subsamples from the **same** data
  - same size, can have **repeated examples**
- ***k*** test sets, made of the examples left out of each bootstrap
  - on average 63.2% of the data set is used per sample
- Obtain an accuracy estimate for each subsample
- Calculate the average and variance of the ***k*** estimates



BOOTSTRAP METHOD

Out-of-Bag (OOB) observations

Each bootstrap sample leaves some original points unused (these are the out-of-bag (OOB) observations):

$$P(\text{point excluded}) = (1 - 1/N)^N \rightarrow e^{-1} \approx 0.368 \quad (\text{as } N \rightarrow \infty)$$

In-bag ≈ 63.2%

Points selected (possibly more than once)
- used to train this bootstrap model.

Out-of-bag ≈ 36.8%

Points never selected
- unused in training, so they're unseen by this model.

Why it matters: OOB points act as a free validation set for that model — no separate CV split needed. This is exactly how the OOB error is computed in Random Forests: each tree is trained on its own bootstrap sample, then validated on its own OOB points, and the errors are combined across trees.

Bootstrapping in code

PYTHON

```
import numpy as np
# Load the breast cancer dataset

from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()

X = data.data # features
y = data.target # labels

# Define the number of bootstrap samples
n_bootstrap = 100

# Define an empty list to store the bootstrap samples
bootstrap_samples = []

# Loop over the number of bootstrap samples
for i in range(n_bootstrap):
   # Draw random indices with replacement
   indices = np.random.choice(len(X), size=len(X), replace=True)
   # Extract the bootstrap sample
   X_bootstrap = X[indices]
   y_bootstrap = y[indices]
   # Append the bootstrap sample to the list
   bootstrap_samples.append((X_bootstrap, y_bootstrap))

#from: https://gpttutorpro.com/machine-learning-evaluation-mastery-how-to-use-bootstrap-for-model-evaluation-and-comparison/
```

Further reading

- Han, Kamber & Pei, Data Mining Concepts and Techniques, Morgan Kaufman.
- Jake VanderPlas, Data Science Handbook, O'Reilly
- Janez Demsar, Statistical Comparisons of Classifiers over Multiple Data Sets, JMLR, 2006

- ***ISLP***
- Statistical Learning: 5.1 Cross Validation
- Statistical Learning: 5.2 K-fold Cross Validation
- Statistical Learning: 5.3 Cross Validation the wrong and right way
- Statistical Learning: 5.4 The Bootstrap
- Statistical Learning: 5.5 More on the Bootstrap
- Statistical Learning: 6.5 Validation and cross validation

- ***StatQuest***
- Fundamentos de aprendizado de máquina: Validação Cruzada
- Bootstrapping Main Ideas!!!

KEY TAKEAWAYS

Wrapping up

- Model validation is essential: we need to estimate performance on unseen data, not just training data
- The bias-variance trade-off defines the sweet spot between under- and over-fitting
- Holdout is simple but high-variance; k-fold CV (typically 5 or 10) trades some bias for much lower variance
- LOOCV (K=N) is nearly unbiased but expensive; stratified CV helps with imbalanced classes
- The bootstrap resamples with replacement to quantify uncertainty — and always keep validation and test sets separate from tuning